



Unit-4



Prepared by, N N Madhak

GTU Syllabus:

Relational Database Design:

- ▶ Features of good relational designs, Dependency preservation, Functional Dependency—definition, Trivial and Non-Trivial FD, Closure of FD set, Closure of attributes, Irreducible set of FD, Decomposition using FD, Normalization – 1NF, 2NF, 3NF, BCNF, Multivalued dependency, 4NF.



Dependency in DBMS:

- ▶ A dependency is a constraint that governs or defines the relationship between two or more attributes.
- ▶ In a database, it happens when information recorded in the same table uniquely determines other information stored in the same table.
- ▶ This may also be described as a relationship in which knowing the value of one attribute (or collection of attributes) in the same table tells you the value of another attribute (or set of attributes).
- ▶ It's critical to understand database dependencies since they serve as the foundation for database normalization.



Functional Dependency in DBMS

- ▶ In a database management system, functional dependence (FD) is a constraint that establishes the relationship between one attribute and another attribute. Functional Dependency aids in preserving the integrity of the database's data.
- ▶ If there is only one non-key attribute in the table, any changes made to the primary key attribute also affect the non-key attribute. If any changes are made to the functional dependencies, the primary key table contents will be affected, and the functional dependencies are said to have a "minimal" functional dependency case



-
- ▶ Determinant refers to the attribute set on the left side of the arrow, X . In contrast, Dependent refers to the attribute set on the right side, Y . While Y will be a dependent non-key attribute from a table that shares the main key with X , X will be the primary key attribute.
 - ▶ Example and its types [Refer from material]
 - ▶ **Armstrong Axioms** [Refer from material]



-
- ▶ In a database management system (DBMS) system, functional Dependency in DBMS establishes the relationship between one attribute and another attribute. Functional dependencies make the quality of the data in the database easier.
 - ▶ The symbol for a functional dependency is an arrow. $X \rightarrow Y$ stands for the functional dependence of X on Y . Functional Dependency is a key factor in determining the quality of database architecture.



Closure of a set of FDs:

What is closure of a set of FDs?

- ▶ Given a set F set of functional dependencies, there are certain other functional dependencies that are logically implied by F .

E.g.: $F = \{A \rightarrow B \text{ and } B \rightarrow C\}$, then we can infer that $A \rightarrow C$ (by transitivity rule)

- ▶ The set of functional dependencies (FDs) that is logically implied by F is called the closure of F .

It is denoted by F^+ .

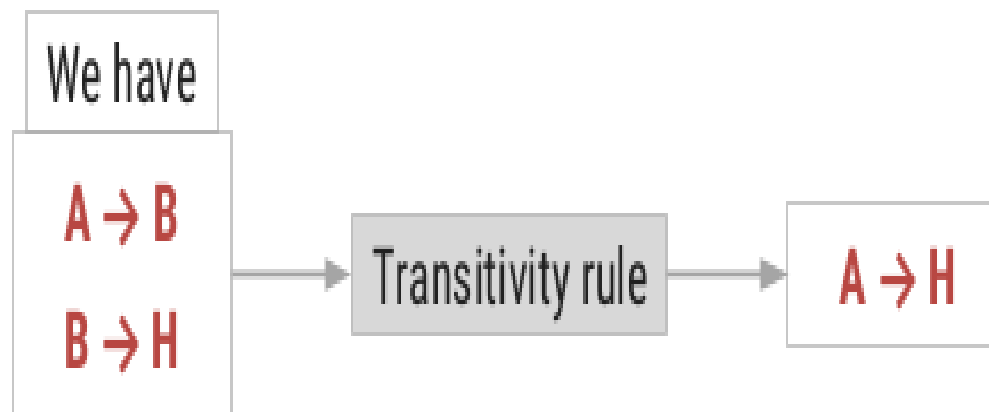


Closure of a set of FDs [Example]

□ Suppose we are given a relation schema $R(A,B,C,G,H,I)$ and the set of functional dependencies are:

□ $F = (\underline{A} \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, \underline{B} \rightarrow H)$

- The functional dependency $A \rightarrow H$ is logical implied.

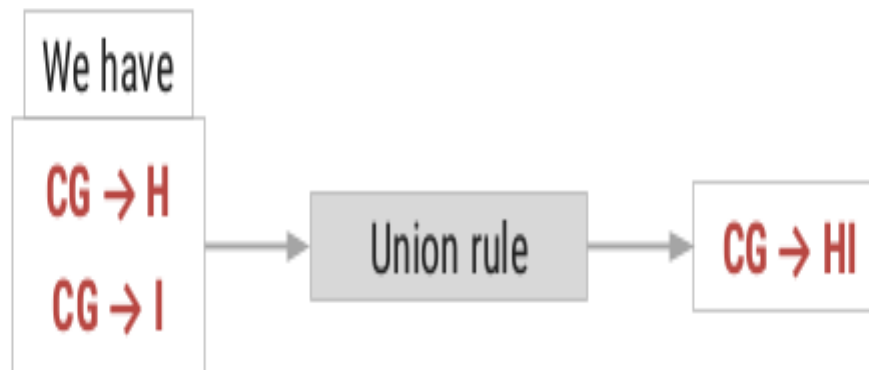


Closure of a set of FDs [Example]

□ Suppose we are given a relation schema $R(A,B,C,G,H,I)$ and the set of functional dependencies are:

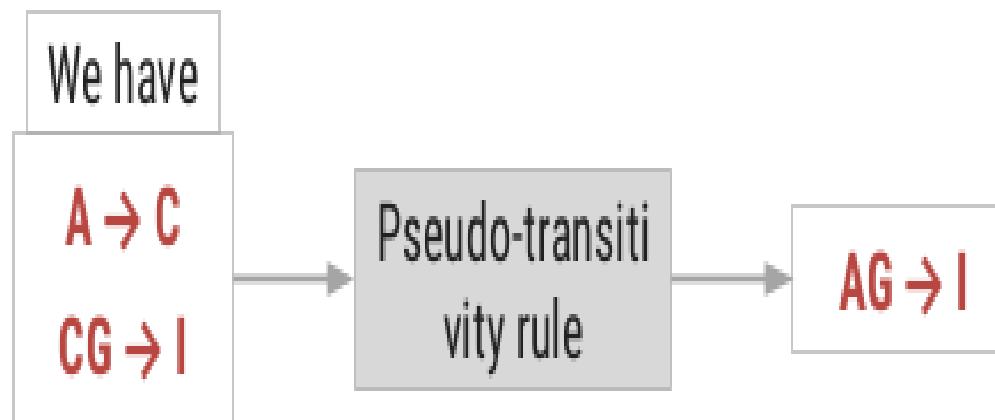
□ $F = (A \rightarrow B, A \rightarrow C, \underline{CG \rightarrow H}, \underline{CG \rightarrow I}, B \rightarrow H)$

- The functional dependency $CG \rightarrow HI$ is logical implied.



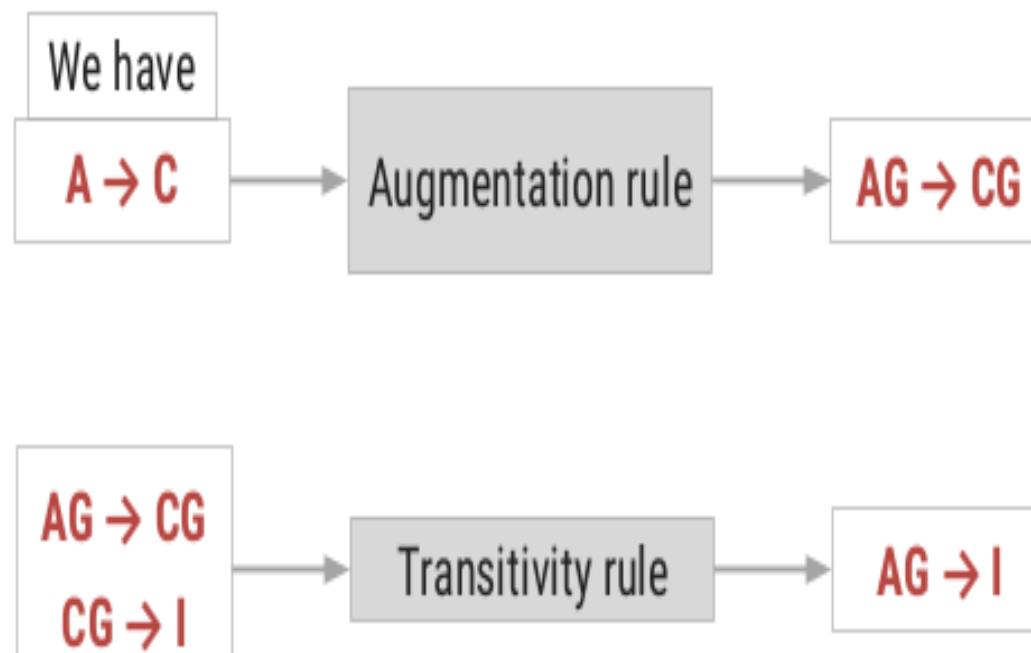
Closure of a set of FDs [Example]

- Suppose we are given a relation schema $R(A,B,C,G,H,I)$ and the set of functional dependencies are:
 - $F = (A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H)$
 - The functional dependency $AG \rightarrow I$ is logical implied.



Closure of a set of FDs [Example]

- Suppose we are given a relation schema $R(A,B,C,G,H,I)$ and the set of functional dependencies are:
 - $F = (A \rightarrow B, \underline{A \rightarrow C}, CG \rightarrow H, \underline{CG \rightarrow I}, B \rightarrow H)$
 - The functional dependency $AG \rightarrow I$ is logical implied.



Closure of a set of FDs [Example]

□ Suppose we are given a relation schema $R(A,B,C,G,H,I)$ and the set of functional dependencies are:

$$\square F = (A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H)$$

- Find out the closure of F .

Several members of F^+ are

$$F^+ = (A \rightarrow H, CG \rightarrow HI, AG \rightarrow I)$$

Closure of a set of FDs [Example]

- Compute the closure of the following set F of functional dependencies FDs for relational schema $R = (A, B, C, D, E, F)$:
 - $F = (A \rightarrow B, A \rightarrow C, CD \rightarrow E, CD \rightarrow F, B \rightarrow E)$
 - Find out the closure of F .

$A \rightarrow B \ \& \ A \rightarrow C$	Union Rule	$A \rightarrow BC$
$CD \rightarrow E \ \& \ CD \rightarrow F$	Union Rule	$CD \rightarrow EF$
$A \rightarrow B \ \& \ B \rightarrow E$	Transitivity Rule	$A \rightarrow E$
$A \rightarrow C \ \& \ CD \rightarrow E$	Pseudo-transitivity Rule	$AD \rightarrow E$
$A \rightarrow C \ \& \ CD \rightarrow F$	Pseudo-transitivity Rule	$AD \rightarrow F$

$$F^+ = (A \rightarrow BC, CD \rightarrow EF, A \rightarrow E, AD \rightarrow E, AD \rightarrow F)$$

Closure of a set of FDs [Example]

□ Compute the closure of the following set F of functional dependencies FDs for relational schema $R = (A, B, C, D, E)$:

□ $F = (AB \rightarrow C, D \rightarrow AC, D \rightarrow E)$

▪ Find out the closure of F .

$D \rightarrow AC$	Decomposition Rule	$D \rightarrow A \ \& \ D \rightarrow C$
$D \rightarrow AC \ \& \ D \rightarrow E$	Union Rule	$D \rightarrow ACE$

$F^+ = (D \rightarrow A, D \rightarrow C, D \rightarrow ACE)$

Closure of attribute sets:

What is a closure of attribute sets?

- ▶ Given a set of attributes α , the closure of α under F is the set of attributes that are functionally determined by α under F . It is denoted by α^+ .



- Given a set of attributes a , the closure of a under F is the **set of attributes that are functionally determined by a under F** .
- It is denoted by a^+ .

Algorithm

- Algorithm to compute a^+ , the closure of a under F
 - Steps
 1. $result = a$
 2. *while* (changes to $result$) *do*
 - for each $\beta \rightarrow \gamma$ in F *do*
 - *begin*
 - if $\beta \subseteq result$ then $result = result \cup \gamma$
 - else $result = result$
 - *end*

Closure of attribute sets [Example]

► Consider the relation schema $R = (A, B, C, G, H, I)$.

For this relation, a set of functional dependencies F can be given as $F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$

Find out the closure of $(AG)^+$.

Algorithm

□ Algorithm to compute a^+ , the closure of a under F

□ Steps

1. $result = a$

2. *while* (changes to $result$) *do*

□ *for each* $\beta \rightarrow \gamma$ *in* F *do*

▪ *begin*

• *if* $\beta \subseteq result$ *then* $result = result \cup \gamma$

• *else* $result = result$

▪ *end*

□ Step 1.

$result = a \Rightarrow result = AG$

$A \rightarrow B$	$A \subseteq AG$	$result = ABG$
$A \rightarrow C$	$A \subseteq ABG$	$result = ABCG$
$CG \rightarrow H$	$CG \subseteq ABCG$	$result = ABCGH$
$CG \rightarrow I$	$CG \subseteq ABCGH$	$result = ABCGHI$
$B \rightarrow H$	$B \subseteq ABCGHI$	$result = ABCGHI$

$AG^+ = ABCGHI$

Closure of attribute sets [Exercise]

- ▶ Given functional dependencies (FDs) for relational schema $R = (A, B, C, D, E): F = \{A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A\}$
- ▶ Find Closure for A
- ▶ Find Closure for CD
- ▶ Find Closure for B
- ▶ Find Closure for BC
- ▶ Find Closure for E

Answer

$A^+ = ABCDE$

$CD^+ = ABCDE$

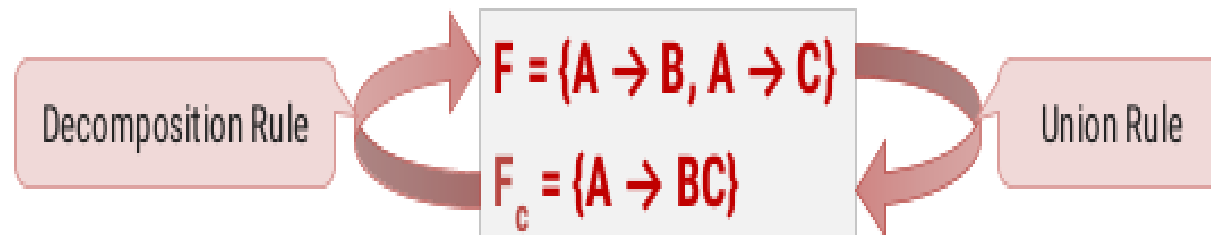
$B^+ = BD$

$BC^+ = ABCDE$

$E^+ = ABCDE$

What is canonical cover?

- A canonical cover of F is a **minimal set of functional dependencies** equivalent to F , having **no redundant dependencies or redundant parts of dependencies**.
- It is denoted by F_c
- A canonical cover for F is a set of dependencies F_c such that
 - F **logically implies** all dependencies in F_c and
 - F_c **logically implies** all dependencies in F and
 - **No** functional dependency in F_c contains an **extraneous attribute** and
 - Each **left side** of functional dependency in F_c is **unique**.



Algorithm to find canonical cover

□ Repeat

□ Use the **union rule** to replace any dependencies in F $\alpha_1 \rightarrow \beta_1$ and $\alpha_1 \rightarrow \beta_2$ with $\alpha_1 \rightarrow \beta_1\beta_2$

□ Find a functional dependency $\alpha \rightarrow \beta$ with an **extraneous attribute** either in α or in β

/* Note: test for extraneous attributes done using F_c , not F */

- If an **extraneous attribute is found, delete it** from $\alpha \rightarrow \beta$

□ until F does not change

/* Note: Union rule may become applicable after some extraneous attributes have been deleted, so it has to be re-applied */



Canonical cover [Example]

- Consider the relation schema $R = (A, B, C)$ with FDs

$$F = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$$

- Find canonical cover.

- Combine $A \rightarrow BC$ and $A \rightarrow B$ into $A \rightarrow BC$ (Union Rule)

- Set is $\{A \rightarrow BC, B \rightarrow C, AB \rightarrow C\}$

- A is extraneous in $AB \rightarrow C$

- Check if the result of deleting A from $AB \rightarrow C$ is implied by the other dependencies
 - Yes: in fact, $B \rightarrow C$ is already present

- Set is $\{A \rightarrow BC, B \rightarrow C\}$

- C is extraneous in $A \rightarrow BC$

- Check if $A \rightarrow C$ is logically implied by $A \rightarrow B$ and the other dependencies
 - Yes: using transitivity on $A \rightarrow B$ and $B \rightarrow C$.

- The canonical cover is: $A \rightarrow B, B \rightarrow C$

Canonical cover [Example]

□ Consider the relation schema $R = (A, B, C, D, E, F)$ with FDs

$$F = \{A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A\}$$

□ Find canonical cover.

□ The left side of each FD in F is unique.

□ Also none of the attributes in the left side or right side of any of the FDs is extraneous.

□ Therefore the canonical cover F_c is equal to F .

□ $F_c = \{A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A\}$

Canonical cover:

- ▶ What is extraneous attributes?
- ▶ Let us consider a relation R with schema $R = (A, B, C)$ and set of functional dependencies FDs $F = \{ AB \rightarrow C, A \rightarrow C \}$.
- ▶ In $AB \rightarrow C$, B is extraneous attribute. The reason is, there is another FD $A \rightarrow C$, which means when A alone can determine C , the use of B is unnecessary (extra).
- ▶ An attribute of a functional dependency is said to be extraneous if we can remove it without changing the closure of the set of functional dependencies.



Decomposition:

- ▶ Decomposition is the process of breaking down given relation into two or more relations.
- ▶ Relation R is replaced by two or more relations in such a way that:
 - Each new relation contains a subset of the attributes of R
 - Together, they all include all tuples and attributes of R

Types of decomposition

- ▶ Lossy decomposition
 - ▶ Lossless decomposition (non-loss decomposition)
-

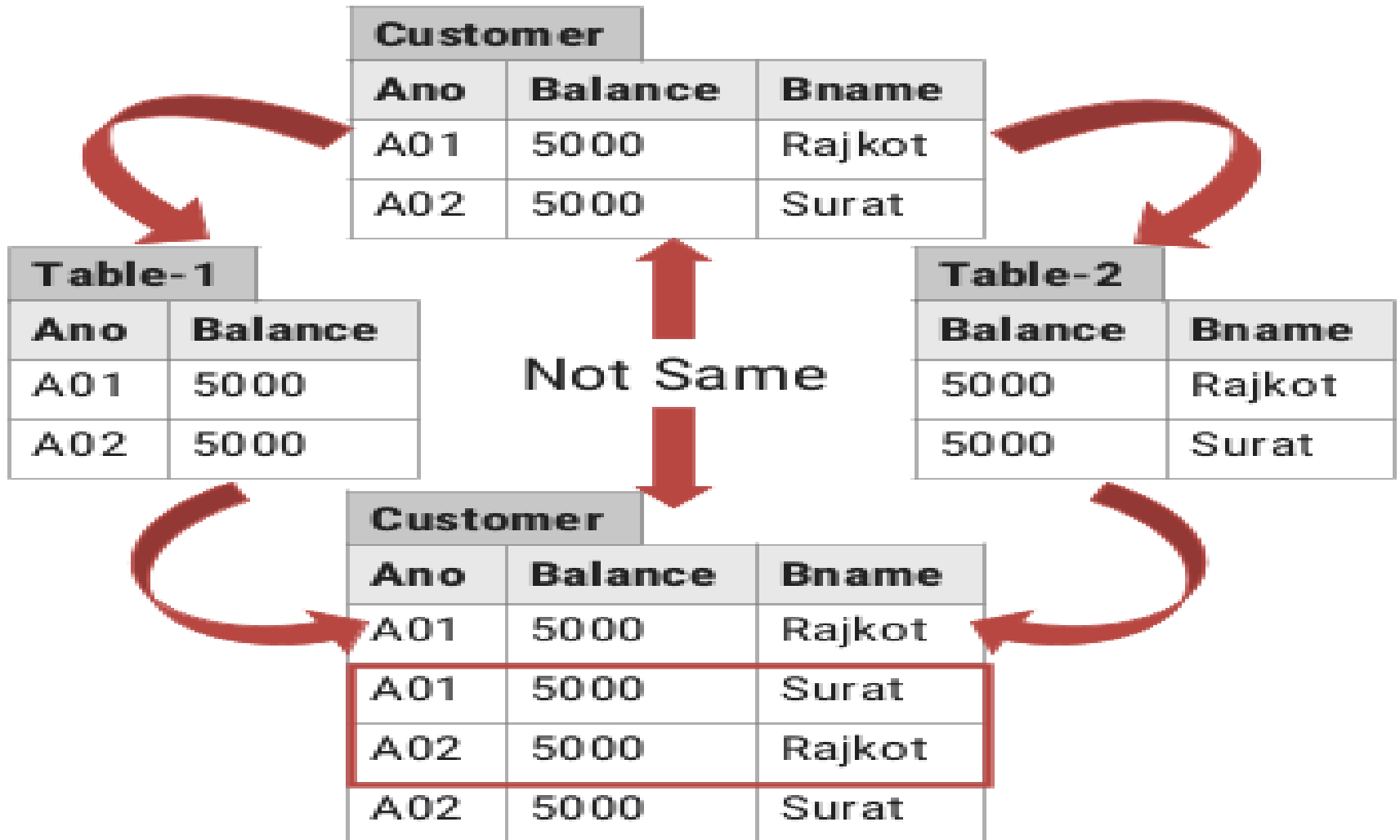


Lossy decomposition:

- ▶ The decomposition of relation R into R_1 and R_2 is lossy when the join of R_1 and R_2 does not yield the same relation as in R .
- ▶ This is also referred as lossy-join decomposition.
- ▶ The disadvantage of such kind of decomposition is that some information is lost during retrieval of original relation.
- ▶ From practical point of view, decomposition should not be lossy decomposition.

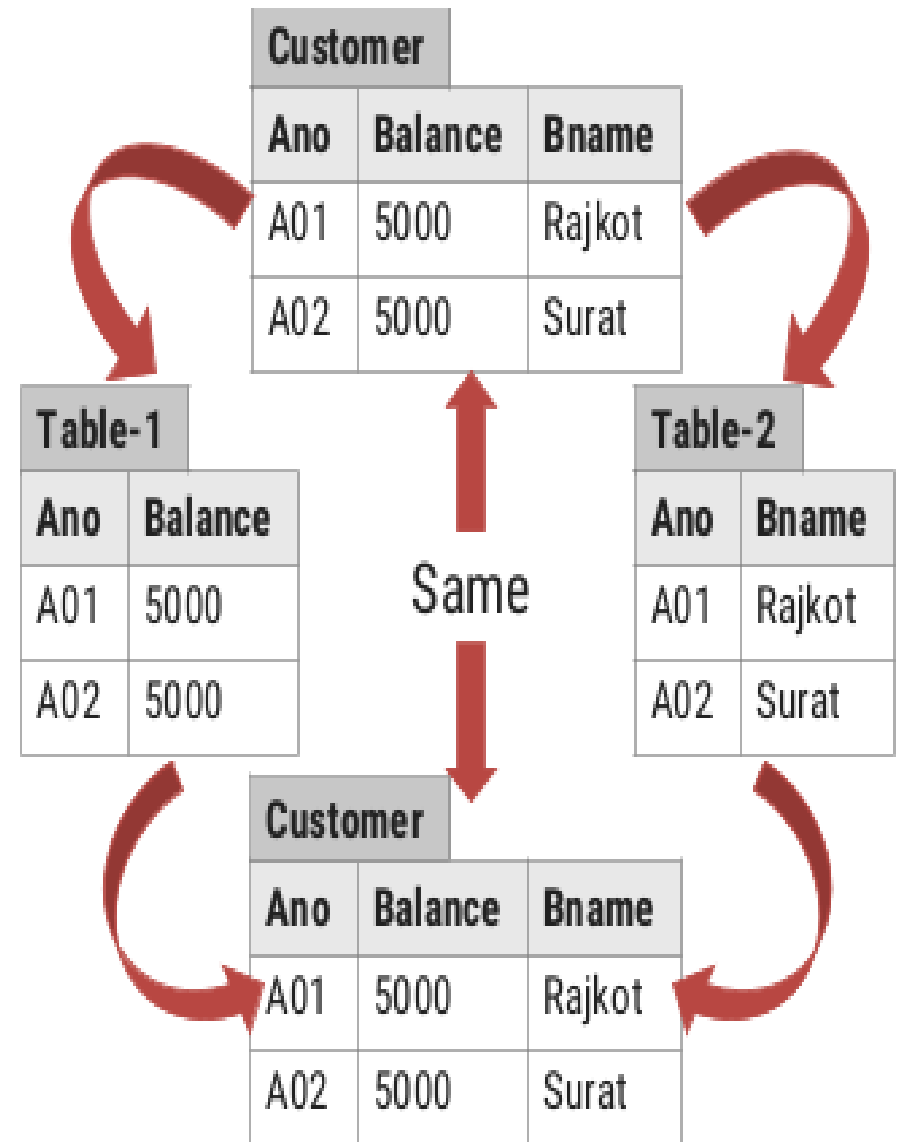


Lossy decomposition:



Lossless decomposition:

- ▶ The decomposition of relation R into R1 and R2 is lossless when the join of R1 and R2 produces the same relation as in R.
- ▶ This is also referred as a non-additive (non-loss) decomposition.
- ▶ All decompositions must be lossless.



Anomaly and its types:

- ▶ Anomalies are problems that can occur in poorly planned, un-normalized database where all the data are stored in one table.

There are three types of anomalies that can arise in the database because of redundancy are

- ▶ Insert anomaly: Problem which can be occurred while inserting record in the database is called insertion anomaly.
- ▶ Deletion anomaly: Problem which can be occurred while deleting record in the database is called deletion anomaly.
- ▶ Update / Modification anomaly: Problem which can be occurred while updating record in the database is called modification anomaly.



Example:

<u>Order No</u>	<u>Order date</u>	<u>Item code</u>	<u>Qty</u>	<u>Price/ur</u>
1456	26021999	3687	52	50.40
1456	26021999	4627	38	60.20
1456	26021999	3214	20	17.50
1886	04031999	4629	45	20.25
1886	04031999	4627	30	60.20
1788	04111999	4627	40	60.20



-
- Insertion Anomaly: Enter new item with code 3945 and price 30.50 for which no order has been placed. Inserted tuple will have no values (i.e have to be left blank) for order no and order date
 - Deletion Anomaly: If order no1886 is deleted the information that item code 4629 costs 20.25 is lost
 - Modification Anomaly: If price of item 4627 is changed, all instances of this item code have to be changed by exhaustive search-errors possible
-



What Normalization Stands for?

- ▶ Normalization is a method of organizing data in a database that helps to reduce data redundancy, insertion, update, and deletion errors. It is the process of assessing relation schemas based on functional relationships and primary keys.
- ▶ This allows you to limit the amount of space a database takes up while also ensuring that the data is kept correctly.



Normalization Need:

- ▶ As previously stated, normalization is used to eliminate data redundancy. It offers a mechanism for removing the following anomalies from the database and making it more consistent:
 - ▶ A database anomaly is a fault in the database caused by insufficient preparation and redundancy.
 - ▶ **Insertion anomalies** occur when we are unable to insert data into a database due to the absence of particular attributes at the moment of insertion.
 - ▶ **Updation anomalies** occur when the same data items with the same values are repeated but are not related to each other.
 - ▶ A **deletion anomaly** happens when deleting one part of the data results in the deletion of the other necessary information from the database.
-



What is normalization?

- ▶ Normalization is the **process of removing redundant data** from tables to **improve data integrity, scalability and storage efficiency**.
 - ▶ data integrity (completeness, accuracy and consistency of data)
 - ▶ scalability (ability of a system to continue to function well in a growing amount of work)
 - ▶ storage efficiency (ability to store and manage data that consumes the least amount of space)
- ▶ What we do in normalization?
 - ▶ Normalization generally involves **splitting an existing table into multiple (more than one) tables**, which can be **re-joined or linked** each time a query is issued (executed).



How many normal forms are there?

- ▶ Normal forms:
 - ▶ 1NF (First normal form)
 - ▶ 2NF (Second normal form)
 - ▶ 3NF (Third normal form)
 - ▶ BCNF (Boyce–Codd normal form)
 - ▶ 4NF (Fourth normal form)
 - ▶ 5NF (Fifth normal form)

As we move from 1NF to 5NF **number of tables** and **complexity increases** but **redundancy decreases**.



Normal forms

1NF (First Normal Form)

- ▶ Conditions for 1NF

Each **cells of a table should contain a single value.**

- ▶ A relation R is in first normal form (1NF) if and only if it **does not contain any composite attribute or multi-valued attributes or their combinations.**

OR

- ▶ A relation R is in first normal form (1NF) if and only if **all underlying domains contain atomic values only.**



Customer		
<u>CID</u>	Name	Address
C01	Raju	Jamnagar Road, Rajkot
C02	Mitesh	Nehru Road, Jamnagar
C03	Jay	C.G Road, Ahmedabad

- In customer relation **address** is **composite attribute** which is further divided into sub-attributes as “Road” and “City”.
- So customer relation is not in 1NF.

Problem: It is **difficult to retrieve the list of customers living in 'Jamnagar' city** from customer table.

The reason is that **address attribute is composite attribute** which **contains road name as well as city name in single cell**.

It is possible that **city name word is also there in road name**.

In our example, 'Jamnagar' word occurs in both records, in first record it is a part of road name and in second one it is the name of city.



Customer

<u>CI D</u>	Name	Address
C01	Raju	Jamnagar Road, Rajkot
C02	Mitesh	Nehru Road, Jamnagar
C03	Jay	C.G Road, Ahmedabad



Customer

<u>CI D</u>	Name	Road	City
C01	Raju	Jamnagar Road	Rajkot
C02	Mitesh	Nehru Road	Jamnagar
C03	Jay	C.G Road	Ahmedabad

Solution: Divide composite attributes into number of sub-attributes and insert value in proper sub-attribute.



1NF (First Normal Form) [Example - Multivalued attribute]

Student		
<u>Rno</u>	Name	FailedinSubjects
101	Raju	DS, DBMs
102	Mitesh	DBMS, DS
103	Jay	DS, DBMS, DE
104	Jeet	DBMS, DE, DS
105	Harsh	DE, DBMS, DS
106	Neel	DE, DBMS

- In student relation **FailedinSubjects attribute is a multi-valued attribute** which can store more than one values.
- So above relation is not in 1NF.

Problem: It is difficult to retrieve the **list of students failed in 'DBMS' as well as 'DS' but not in other subjects** from student table.

The reason is that FailedinSubjects attribute is multi-valued attribute so it contains more than one value.



Student					Result		
<u>Rno</u>	Name	FailedinSubjects			<u>RID</u>	Rno	Subject
101	Raju	DS, DBMs			1	101	DS
102	Mitesh	DBMS, DS			2	101	DBMS
103	Jay	DS, DBMS, DE			3	102	DBMS
104	Jeet	DBMS, DE, DS			4	102	DS
105	Harsh	DE, DBMS, DS			5	103	DS
106	Neel	DE, DBMS			...	...	...

Solution: Split the table into two tables in such as way that
the **first table contains all attributes except multi-valued attribute** with
same primary key and
second table contains multi-valued attribute and place a primary key in
it.
insert the primary key of first table in the second table as a foreign key.

2NF (Second Normal Form)

- ▶ Conditions for 2NF

It is in **1NF** and each **table should contain a single primary key**

- ▶ A relation R is in second normal form (2NF)
 - ▶ if and only if it is in **1NF** and
 - ▶ **every non-primary key attribute is fully dependent on the primary key**

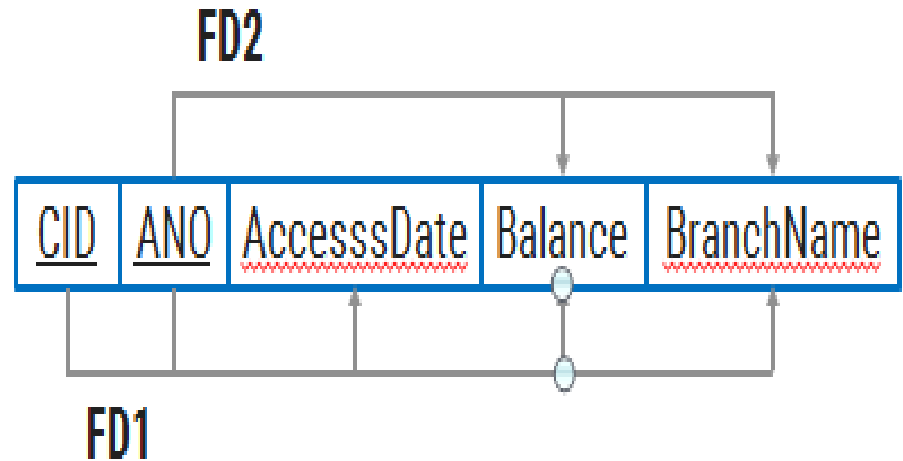
OR

- ▶ A relation R is in second normal form (2NF)
 - ▶ if and only if it is in **1NF** and
 - ▶ **no any non-primary key attribute is partially dependent on the primary key**



2NF (Second Normal Form) [Example]

Customer				
<u>CID</u>	<u>ANO</u>	<u>AccessDate</u>	Balance	<u>BranchName</u>
C01	A01	01-01-2017	50000	Rajkot
C02	A01	01-03-2017	50000	Rajkot
C01	A02	01-05-2017	25000	Surat
C03	A02	01-07-2017	25000	Surat



FD1: {CID, ANO} → {AccesssDate, Balance, BranchName}

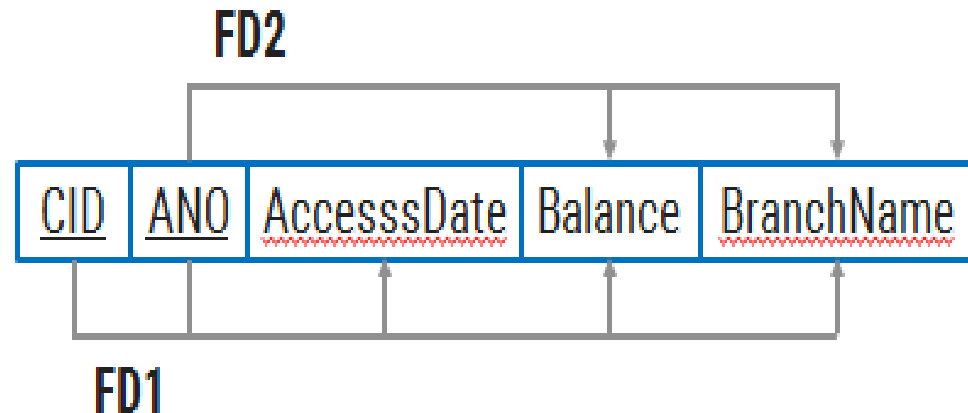
FD2: ANO → {Balance, BranchName}

Balance and BranchName are partial dependent on primary key (CID + ANO).

So customer relation is not in 2NF.

2NF (Second Normal Form) [Example]

Customer				
<u>CID</u>	<u>ANO</u>	<u>AccessDate</u>	Balance	<u>BranchName</u>
C01	A01	01-01-2017	50000	Rajkot
C02	A01	01-03-2017	50000	Rajkot
C01	A02	01-05-2017	25000	Surat
C03	A02	01-07-2017	25000	Surat



Problem: For example, in case of a joint account multiple (more than one) customers have common (one) accounts.

If an account '**A01**' is operated jointly by two customers says '**C01**' and '**C02**' then data values for attributes **Balance** and **BranchName** will be duplicated in two different tuples of customers '**C01**' and '**C02**'.

Customer				
<u>CID</u>	<u>ANO</u>	<u>AccessDate</u>	Balance	<u>BranchName</u>
C01	A01	01-01-2017	50000	Rajkot
C02	A01	01-03-2017	50000	Rajkot
C01	A02	01-05-2017	25000	<u>Surat</u>
C03	A02	01-07-2017	25000	<u>Surat</u>



Table-1		
<u>ANO</u>	Balance	<u>BranchName</u>
A01	50000	Rajkot
A02	25000	<u>Surat</u>

Table-2		
<u>CID</u>	<u>ANO</u>	<u>AccessDate</u>
C01	A01	01-01-2017
C02	A01	01-03-2017
C01	A02	01-05-2017
C03	A02	01-07-2017

Solution: Decompose relation in such a way that **resultant relations do not have any partial FD.**

Remove partial dependent attributes from the relation that violates 2NF.

Place them in separate relation along with the **prime attribute on which they are fully dependent.**

The **primary key of new relation** will be the **attribute on which it is fully dependent.**

Keep other attributes same as in that table with the **same primary key.**

Normal forms

3NF (Third Normal Form)

- ▶ Conditions for 3NF
- ▶ It is in 2NF and there is no transitive dependency.
- ▶ (Transitive dependency???) $A \rightarrow B$ & $B \rightarrow C$ then $A \rightarrow C$
- ▶ A relation R is in third normal form (3NF)
 - ▶ if and only if it is in **2NF** and
 - ▶ **every non-key attribute is non-transitively dependent on the primary key**

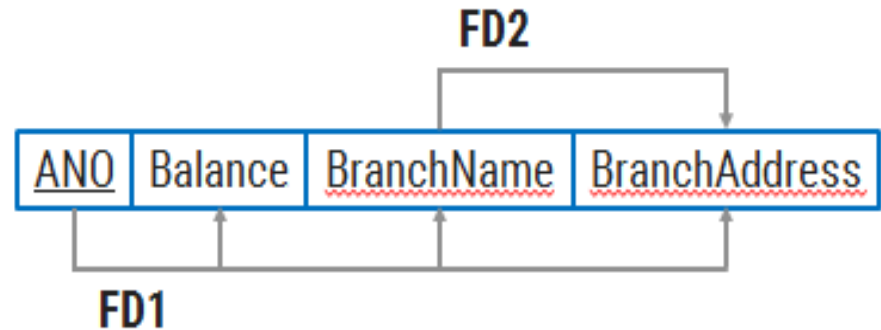
OR

- ▶ A relation R is in third normal form (3NF)
 - ▶ if and only if it is in **2NF** and
 - ▶ **no any non-key attribute is transitively dependent on the primary key**
-



3NF (Third Normal Form) [Example]

Customer			
<u>ANO</u>	Balance	<u>BranchName</u>	<u>BranchAddress</u>
A01	50000	Rajkot	Kalawad road
A02	40000	Rajkot	Kalawad Road
A03	35000	<u>Surat</u>	C.G Road
A04	25000	<u>Surat</u>	C.G Road



FD1: $ANO \rightarrow \{Balance, BranchName, BranchAddress\}$

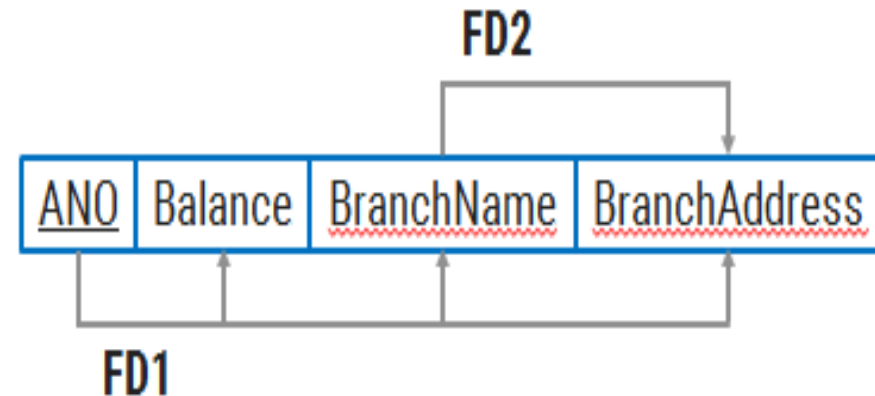
FD2: $BranchName \twoheadrightarrow BranchAddress$

So AccountNO $\rightarrow$ BranchAddress (Using Transitivity rule)

BranchAddress is transitively dependent on primary key (ANO). So customer relation is not in 3NF.

3NF (Third Normal Form) [Example]

Customer			
<u>ANO</u>	Balance	<u>BranchName</u>	<u>BranchAddress</u>
A01	50000	Rajkot	Kalawad road
A02	40000	Rajkot	Kalawad Road
A03	35000	<u>Surat</u>	C.G Road
A04	25000	<u>Surat</u>	C.G Road



Problem: In this relation, **branch address will be stored repeatedly** for each account of the same branch which **occupies more space**.

Customer			
<u>ANO</u>	Balance	<u>BranchName</u>	<u>BranchAddress</u>
A01	50000	Rajkot	Kalawad road
A02	40000	Rajkot	Kalawad Road
A03	35000	Surat	C.G Road
A04	25000	Surat	C.G Road



Table-1	
<u>BranchName</u>	<u>BranchAddress</u>
Rajkot	Kalawad road
Surat	C.G Road

Table-2		
<u>ANO</u>	Balance	<u>BranchName</u>
A01	50000	Rajkot
A02	40000	Rajkot
A03	35000	Surat
A04	25000	Surat

Solution: Decompose relation in such a way that **resultant relations do not have any transitive FD.**

Remove transitive dependent attributes from the relation that violates 3NF.

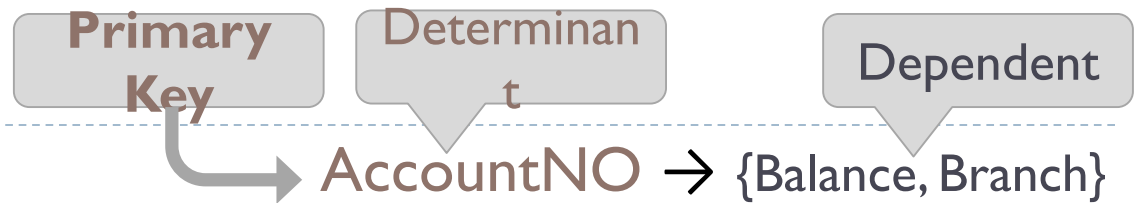
Place them in a new relation along with the **non-prime attributes** due to which transitive dependency occurred.

The **primary key** of the new relation will be **non-prime attributes** due to which transitive dependency occurred.

Keep other attributes same as in the table with same **primary key** and add **prime attributes** of other relation into it as a **foreign key**.



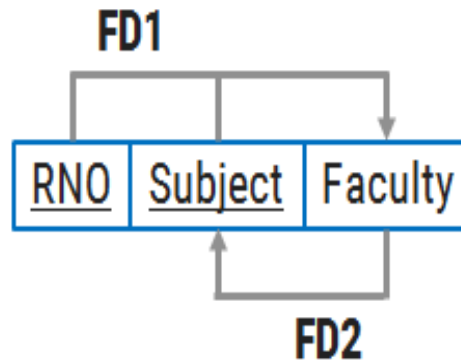
BCNF (Boyce-Codd Normal Form)



- ▶ Conditions for BCNF:
 - ▶ BCNF is based on the concept of a determinant
 - ▶ It is in 3NF and every determinant should be primary key.
-
- ▶ A relation R is in Boyce-Codd normal form (BCNF)
 - ▶ if and only if it is in 3NF and
 - ▶ for every functional dependency $X \rightarrow Y$, X should be the primary key of the table.**OR**
 - ▶ A relation R is in Boyce-Codd normal form (BCNF)
 - ▶ if and only if it is in 3NF and
 - ▶ every prime key attribute is non-transitively dependent on the primary key**OR**
 - ▶ A relation R is in Boyce-Codd normal form (BCNF)
 - ▶ if and only if it is in 3NF and
-
- ▶ ▶ no any prime key attribute is transitively dependent on the primary key

BCNF Example:

Student		
<u>RNO</u>	<u>Subject</u>	Faculty
101	DS	Patel
102	DBMS	Shah
103	DS	Jadeja
104	DBMS	Dave
105	DBMS	Shah
102	DS	Patel
101	DBMS	Dave
105	DS	<u>Jadeja</u>



- **FD1:** $RNO, Subject \rightarrow Faculty$
- **FD2:** $Faculty \rightarrow Subject$
- So $\{RNO, Subject\} \rightarrow Subject$ (Transitivity rule)

In FD2, **determinant is Faculty which is not a primary key**. So student table is not in BCNF.

Problem: In this relation **one student can learn more than one subject with different faculty** then **records will be stored repeatedly for each student, language and faculty combination** which **occupies more space**.

- Here, one faculty teaches only one subject, but a subject may be taught by more than one faculty.
- A student can learn a subject from only one faculty.

Student		
<u>RNO</u>	<u>Subject</u>	Faculty
101	DS	Patel
102	DBMS	Shah
103	DS	Jadeja
104	DBMS	Dave
105	DBMS	Shah
102	DS	Patel
101	DBMS	Dave
105	DS	<u>Jadeja</u>



Table-1	
<u>Faculty</u>	<u>Subject</u>
Patel	DS
Shah	DBMS
Jadeja	DS
Dave	DBMS

Table-2	
<u>RNO</u>	<u>Faculty</u>
101	Patel
102	Shah
103	Jadeja
104	Dave
105	Shah
102	Patel
101	Dave
105	<u>Jadeja</u>

- **Solution:** Decompose relation in such a way that resultant relations do not have any transitive FD.
 - Remove transitive dependent prime attribute from relation that violates BCNF.
 - Place them in separate new relation along with the non-prime attribute due to which transitive dependency occurred.
 - The primary key of new relation will be this non-prime attribute due to which transitive dependency occurred.
 - Keep other attributes same as in that table with same primary key and add a prime attribute of other relation into it as a foreign key.

Multivalued dependency (MVD)

- ▶ For a dependency $X \rightarrow Y$, if **for a single value of X, multiple values of Y exists**, then the **table may have multi-valued dependency**.
- ▶ Multivalued dependency (MVD) is denoted by $\rightarrow\rightarrow$
- ▶ Multivalued dependency (MVD) is represented as $X \rightarrow\rightarrow Y$

Student		
<u>RNO</u>	<u>Subject</u>	<u>Faculty</u>
101	DS	Patel
101	DBMS	Patel
101	DS	Shah
101	DBMS	Shah

4NF (Forth Normal Form)

- ▶ Conditions for 4NF
- ▶ A relation R is in fourth normal form (4NF)
 - ▶ if and only if it is in **BCNF** and
 - ▶ **has no multivalued dependencies**

Student		
<u>RNO</u>	<u>Subject</u>	<u>Faculty</u>
101	DS	Patel
101	DBMS	Patel
101	DS	Shah
101	DBMS	Shah



Subject	
<u>RNO</u>	<u>Subject</u>
101	DS
101	DBMS

Faculty	
<u>RNO</u>	<u>Faculty</u>
101	Patel
101	Shah

Above student table has multivalued dependency. So student table is not in 4NF.



Functional dependency & Multivalued dependency

- ▶ A table can have both functional dependency as well as multi-valued dependency together.
 - ▶ $RNO \rightarrow \text{Address}$
 - ▶ $RNO \twoheadrightarrow \text{Subject}$
 - ▶ $RNO \twoheadrightarrow \text{Faculty}$

Student			
<u>RNO</u>	<u>Address</u>	<u>Subject</u>	<u>Faculty</u>
101	C. G. Road, Rajkot	DS	Patel
101	C. G. Road, Rajkot	DBMS	Patel
101	C. G. Road, Rajkot	DS	Shah
101	C. G. Road, Rajkot	DBMS	Shah



Subject	
<u>RNO</u>	<u>Subject</u>
101	DS
101	DBMS

Faculty	
<u>RNO</u>	<u>Faculty</u>
101	Patel
101	Shah

Address	
<u>RNO</u>	<u>Address</u>
101	C. G. Road, Rajkot

5NF (Fifth Normal Form)

► Conditions for 5NF

- A relation R is in fifth normal form (5NF)
 - if and only if it is in **4NF** and
 - it **cannot have a lossless decomposition in to any number of smaller tables** (relations).

Student_Result				
<u>RID</u>	RNO	Name	Subject	Result
1	101	Raj	DBMS	Pass
2	101	Raj	DS	Pass
3	101	Raj	DF	Pass
4	102	Meet	DBMS	Pass
5	102	Meet	DS	Fail
6	102	Meet	DF	Pass
7	103	Suresh	DBMS	Fail
8	103	Suresh	DS	Pass

Student_Result relation is **further decomposed** into sub-relations. So the above relation is **not in 5NF**.

5NF

- ▶ Conditions for 5NF
- ▶ A relation R is in fifth normal form (5NF)
 - ▶ if and only if it is in **4NF** and
 - ▶ it **cannot have a lossless decomposition in to any number of smaller tables** (relations).

Student_Result					Student		Subject		Result			
<u>RID</u>	RNO	Name	Subject	Result	<u>RNO</u>	Name	<u>SID</u>	Name	<u>RID</u>	RNO	SID	Result
1	101	Raj	DBMS	Pass	101	Raj	1	DBMS	1	101	1	Pass
2	101	Raj	DS	Pass	102	Meet	2	DS	2	101	2	Pass
3	101	Raj	DF	Pass	103	Suresh	3	DF	3	101	3	Pass
4	102	Meet	DBMS	Pass					4	102	1	Pass
5	102	Meet	DS	Fail					5	102	2	Fail
6	102	Meet	DF	Pass					6	102	3	Pass
7	103	Suresh	DBMS	Fail					7	103	1	Fail
8	103	Suresh	DS	Pass					8	103	2	Pass

None of the above relations can be further decomposed into sub-relations. So the above database is in 5NF.

How to find key?

- ▶ Conditions to find key
 - ▶ The attribute is a **part of key**, if it **does not occur on any side of FD**
 - ▶ The attribute is a **part of key**, if it **occurs on the left-hand side of an FD**, but **never occurs on the right-hand side**
 - ▶ The attribute is **not a part of key**, if it **occurs on the right-hand side of an FD**, but **never occurs on the left-hand side**
 - ▶ The attribute **may be a part of key or not**, if it **occurs on the both side of an FD**



How to find key? [Example]

- ▶ Let a relation R with attributes ABCD with FDs $C \rightarrow A$, $B \rightarrow C$. Find keys for relation R.
 - ▶ attribute **not occur on any side** of FDs (**D**) ✓
 - ▶ attribute **occurs on only left-hand side** of an FDs (**B**) ✓
 - ▶ attribute **occurs on only right-hand side** of an FDs (**A**) X
 - ▶ attribute **occurs on both the sides** of an FDs (**C**) ?
- ▶ The **core is BD**.
- ▶ **B determines C** and **C determines A**, So using **transitivity rule B determines A** also.
- ▶ So **BD is a key**.



Find key?

- ▶ Let a relation R with attributes ABCD with FDs $C \rightarrow D$, $C \rightarrow A$ and $B \rightarrow C$. Find keys for relation R.
 - ▶ The core is B. B determines C which determines A and D, so **B is a key**. Therefore B is the key.
- ▶ Let a relation R with attributes ABCD with FDs $B \rightarrow C$, $D \rightarrow A$. Find keys for relation R.
 - ▶ The core is BD. B determines C and D determines A, so **BD is a key**. Therefore BD is the key.
- ▶ Let a relation R with attributes ABCD with FDs $A \rightarrow B$, $BC \rightarrow D$ and $A \rightarrow C$. Find keys for relation R.
 - ▶ The core is A. A determines B and C which determine D, so **A is a key**. Therefore A is the key.



Find (candidate) key & check for normal forms [Example]

- ▶ Suppose you are given a relation R with four attributes ABCD. For each of the following sets of FDs, do the following: **$F = (B \rightarrow C, D \rightarrow A)$**
- ▶ Identify the candidate key(s) for R.
- ▶ Identify the best normal form that R satisfies (1NF, 2NF, 3NF or BCNF).

Candidate Key is **BD**

Relation R is in **1NF but not 2NF**. In above FDs, **there is a partial dependency**

(As per FD $B \rightarrow C$, **C depends only on B** but **Key is BD** so **C is partial depends on key (BD)**)

(As per FD $D \rightarrow A$, **A depends only on D** but **Key is BD** so **A is partial depends on key (BD)**)

Find (candidate) key & check for normal forms [Example]

- ▶ Suppose you are given a relation R with four attributes ABCD. For each of the following sets of FDs, do the following: **$F = (C \rightarrow D, C \rightarrow A, B \rightarrow C)$**
- ▶ Identify the candidate key(s) for R.
- ▶ Identify the best normal form that R satisfies (1NF, 2NF, 3NF or BCNF).

Candidate Key is **B**

Relation R is in **2NF but not 3NF**. In above FDs, **there is a transitive dependency**
(As per FDs **$B \rightarrow C$ & $C \rightarrow D$** then **$B \rightarrow D$** so **D is transitive depends on key (B)**)
(As per FDs **$B \rightarrow C$ & $C \rightarrow A$** then **$B \rightarrow A$** so **A is transitive depends on key (B)**)

Find (candidate) key & check for normal forms [Example]

- ▶ Suppose you are given a relation R with four attributes ABCD. For each of the following sets of FDs, do the following: **$F = (A \rightarrow B, BC \rightarrow D, A \rightarrow C)$**
- ▶ Identify the candidate key(s) for R.
- ▶ Identify the best normal form that R satisfies (1NF, 2NF, 3NF or BCNF).

Candidate Key is **A**

Relation R is in **2NF but not 3NF**. In above FDs, **there is a transitive dependency**
(As per FDs **$A \rightarrow B$ & $A \rightarrow C$** then **$A \rightarrow BC$** using **union rule**) and
(As per FDs **$A \rightarrow BC$ & $BC \rightarrow D$** then **$A \rightarrow D$** so **D is transitive depends on key (A)**)

Find (candidate) key & check for normal forms [Example]

- ▶ Suppose you are given a relation R with four attributes ABCD. For each of the following sets of FDs, do the following: **F = (ABC → D, D → A)**
- ▶ Identify the candidate key(s) for R.
- ▶ Identify the best normal form that R satisfies (1NF, 2NF, 3NF or BCNF).

Candidate Key are **ABC & BCD**

Relation R is in **3NF but not BCNF**.

In the above FDs, both FDs have **prime attribute (D and A) in dependent (right) side**.



How to normalize database?

- ▶ A software contract and consultancy firm maintains details of all the various projects in which its employees are currently involved. These details comprise: Employee Number, Employee Name, Date of Birth, Department Code, Department Name, Project Code, Project Description, Project Supervisor.
 - ▶ Assume the following:
 - ▶ Each employee number is unique.
 - ▶ Each department has a single department code.
 - ▶ Each project has a single code and supervisor.
 - ▶ Each employee may work on one or more projects.
 - ▶ Employee names need not necessarily be unique.
 - ▶ Project Code, Project Description and Project Supervisor are repeating fields.
 - ▶ Normalize this data to Third Normal Form.
-



How to normalize database?

- ▶ A software contract and consultancy firm maintains details of all the various projects in which its employees are currently involved. These details comprise: Employee Number, Employee Name, Date of Birth, Department Code, Department Name, Project Code, Project Description, Project Supervisor.

UNF

Employee Number	Employee Name	Date of Birth	Department Code	Department Name	Project Code	Project Description	Project Supervisor
1	Raj	1-1-85	1	CE	1	IOT	Patel
2	Meet	4-4-86	2	EC	2	PHP	Shah
3	Suresh	2-2-85	1	CE	1	IOT	Patel
1	Raj	1-1-85	1	CE	2	PHP	Shah

How to normalize database?

UNF

Employee Number	Employee Name	Date of Birth	Department Code	Department Name	Project Code	Project Description	Project Supervisor
1	Raj	1-1-85	1	CE	1	IOT	Patel
2	Meet	4-4-86	2	EC	2	PHP	Shah
3	Suresh	2-2-85	1	CE	1	IOT	Patel
1	Raj	1-1-85	1	CE	2	PHP	Shah

INF

<u>Employee Number</u>	Employee Name	Date of Birth	Department Code	Department Name	<u>Employee Number</u>	<u>Project Code</u>	Project Description	Project Supervisor
1	Raj	1-1-85	1	CE	1	1	IOT	Patel
2	Meet	4-4-86	2	EC	2	2	PHP	Shah
3	Suresh	2-2-85	1	CE	3	1	IOT	Patel
					1	2	PHP	Shah

How to normalize database?

1NF

<u>Employee Number</u>	Employee Name	Date of Birth	Department Code	Department Name
1	Raj	1-1-85	1	CE
2	Meet	4-4-86	2	EC
3	Suresh	2-2-85	1	CE

<u>Employee Number</u>	<u>Project Code</u>	Project Description	Project Supervisor
1	1	IOT	Patel
2	2	PHP	Shah
3	1	IOT	Patel
1	2	PHP	Shah

2NF

<u>Employee Number</u>	Employee Name	Date of Birth	Department Code	Department Name
1	Raj	1-1-85	1	CE
2	Meet	4-4-86	2	EC
3	Suresh	2-2-85	1	CE

<u>Project Code</u>	Project Description	Project Supervisor
1	IOT	Patel
2	PHP	Shah

<u>Employee Number</u>	<u>Project Code</u>
1	1
2	2
3	1
1	2



How to normalize database?

3NF

<u>Employee Number</u>	Employee Name	Date of Birth	Department Code
1	Raj	1-1-85	1
2	Meet	4-4-86	2
3	Suresh	2-2-85	1

<u>Department Code</u>	Department Name
1	CE
2	EC

<u>Project Code</u>	Project Description	Project Supervisor
1	IOT	Patel
2	PHP	Shah

<u>Employee Number</u>	<u>Project Code</u>
1	1
2	2
3	1
1	2



Dependency preservation

- ▶ It is an important constraint of the database.
- ▶ In the dependency preservation, at least one decomposed table must satisfy every dependency.
- ▶ If a relation R is decomposed into relation R_1 and R_2 , then the dependencies of R either must be a part of R_1 or R_2 or must be derivable from the combination of functional dependencies of R_1 and R_2 .
- ▶ For example, suppose there is a relation $R(A, B, C, D)$ with functional dependency set $(A \rightarrow BC)$. The relational R is decomposed into $R_1(ABC)$ and $R_2(AD)$ which is dependency preserving because FD $A \rightarrow BC$ is a part of relation $R_1(ABC)$.



► **Dependency Preservation: A Decomposition**

$D = \{ R_1, R_2, R_3 \dots R_n \}$ of R is dependency preserving wrt a set F of Functional dependency if

► **$(F_1 \cup F_2 \cup \dots \cup F_m)^+ = F^+.$**

► Consider a relation R

► $R \rightarrow F$ {...with some functional dependency(FD)....} R is decomposed or divided into R_1 with FD $\{ f_1 \}$ and R_2 with $\{ f_2 \}$, then there can be three cases:

► **$f_1 \cup f_2 = F$** -----> Decomposition is dependency preserving.

► **$f_1 \cup f_2$** is a subset of F -----> Not Dependency preserving.

► **$f_1 \cup f_2$** is a super set of F -----> This case is not possible.



► **Problem:**

- Let a relation $R(A, B, C, D)$ and functional dependency $\{AB \rightarrow C, C \rightarrow D, D \rightarrow A\}$. Relation R is decomposed into $R_1(A, B, C)$ and $R_2(C, D)$. Check whether decomposition is dependency preserving or not. [Answer Refer from material]

